

## exec\_with\_mask\_1

April 15, 2025

An Exception was encountered at 'In [1]':

Execution using papermill encountered an exception here and stopped:

```
[1]: import numpy as np
import torch
import tensorly as tl
from tensorly.decomposition import parafac
import sparse
import pickle
import os

assert os.path.exists('sptensor.pkl'), 'No such file.'
with open('sptensor.pkl', 'rb') as f:
    data = pickle.load(f)
data = data[:, :, :, :12, :]
expected_shape = data.shape
n_components = 10

# -----
# 3. NumPy-based BPTF (Aaron's original implementation)
# -----
# Import Aaron's BPTF (which uses NumPy/sparse.COO);
# ensure that the bptf package is in your PYTHONPATH.
from bptf import BPTF as BPTF
import bptf

# Create the same data as a NumPy array and a corresponding binary mask
data_np = data
mask_np = np.zeros(expected_shape, dtype=int)
mask_np[:, :, :, 3, :] = 1
mask_np = sparse.COO((mask_np.copy()).astype(np.int64))

# Instantiate and fit the NumPy-based BPTF model.
# Note: This version uses its own preprocess() function and can work with
↳ sparse.COO.
model_np = BPTF(data_shape=data_np.shape, n_components=n_components)
model_np.fit(data, mask=mask_np, max_iter=50, verbose=True, missing_val=1)
```

```
# Reconstruct using arithmetic expectation
reconstruction_np = model_np.reconstruct(mask=None, fill_value=1,
↳drop_diag=False, style='arithmetic')
frobenius_diff_np = np.sqrt(np.sum((data.todense() - reconstruction_np)**2))
print("NumPy BPTF reconstruction Frobenius norm difference:", frobenius_diff_np)
```

ITERATION 0:      Time: 0.000000   Objective: -569847001.48            Change: nan

```
0%|
| 0/50 [00:00<?, ?it/s]

Are the shape parameters positive? True
Are the rate parameters positive? True
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 0]   min uttkrp_DK = 1.658e+05
[mode 0]   min new rate = 1.658e+05   negatives = 0
Is G_DK_M finite before updating? True
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? True

Are the shape parameters positive? True
Are the rate parameters positive? True
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 1]   min uttkrp_DK = 2.151e+03
[mode 1]   min new rate = 2.151e+03   negatives = 0
Is G_DK_M finite before updating? True
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? True

Are the shape parameters positive? True
Are the rate parameters positive? True
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 2]   min uttkrp_DK = 2.735e+04
[mode 2]   min new rate = 2.735e+04   negatives = 0
Is G_DK_M finite before updating? True
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? True

Are the shape parameters positive? True
Are the rate parameters positive? True
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 3]   min uttkrp_DK = -5.210e-09
[mode 3]   min new rate = 9.826e-02   negatives = 0
Is G_DK_M finite before updating? True
```

Are there NaNs in G\_DK\_M after updating cache? False  
Is G\_DK\_M finite after updating? True

Are the shape parameters positive? True  
Are the rate parameters positive? True  
Are the shape parameters finite? True  
Are the rate parameters finite? True  
[mode 4] min uttkrp\_DK = 8.551e+04  
[mode 4] min new rate = 8.551e+04 negatives = 0  
Is G\_DK\_M finite before updating? True  
Are there NaNs in G\_DK\_M after updating cache? False  
Is G\_DK\_M finite after updating? True

2%|  
| 1/50 [00:06<05:35, 6.84s/it]

ITERATION 1: Time: 6.843604 Objective: 18911520.33 Change: 1.03319e+00

Are the shape parameters positive? True  
Are the rate parameters positive? True  
Are the shape parameters finite? True  
Are the rate parameters finite? True  
[mode 0] min uttkrp\_DK = 7.777e+04  
[mode 0] min new rate = 7.777e+04 negatives = 0  
Is G\_DK\_M finite before updating? True  
Are there NaNs in G\_DK\_M after updating cache? False  
Is G\_DK\_M finite after updating? True

Are the shape parameters positive? True  
Are the rate parameters positive? True  
Are the shape parameters finite? True  
Are the rate parameters finite? True  
[mode 1] min uttkrp\_DK = 1.056e+03  
[mode 1] min new rate = 1.056e+03 negatives = 0  
Is G\_DK\_M finite before updating? True  
Are there NaNs in G\_DK\_M after updating cache? False  
Is G\_DK\_M finite after updating? True

Are the shape parameters positive? True  
Are the rate parameters positive? True  
Are the shape parameters finite? True  
Are the rate parameters finite? True  
[mode 2] min uttkrp\_DK = 1.530e+04  
[mode 2] min new rate = 1.530e+04 negatives = 0  
Is G\_DK\_M finite before updating? True  
Are there NaNs in G\_DK\_M after updating cache? False  
Is G\_DK\_M finite after updating? True

Are the shape parameters positive? True  
Are the rate parameters positive? True

Are the shape parameters finite? True  
Are the rate parameters finite? True  
[mode 3] min uttkrp\_DK = -1.266e-09  
[mode 3] min new rate = 1.655e-06 negatives = 0  
Is G\_DK\_M finite before updating? True  
Are there NaNs in G\_DK\_M after updating cache? False  
Is G\_DK\_M finite after updating? True

Are the shape parameters positive? True  
Are the rate parameters positive? True  
Are the shape parameters finite? True  
Are the rate parameters finite? True  
[mode 4] min uttkrp\_DK = 8.720e+04  
[mode 4] min new rate = 8.720e+04 negatives = 0  
Is G\_DK\_M finite before updating? True  
Are there NaNs in G\_DK\_M after updating cache? False  
Is G\_DK\_M finite after updating? True

4%|  
| 2/50 [00:13<05:29, 6.87s/it]

ITERATION 2: Time: 6.884680 Objective: 27200451.64 Change: 4.38301e-01

Are the shape parameters positive? True  
Are the rate parameters positive? True  
Are the shape parameters finite? True  
Are the rate parameters finite? True  
[mode 0] min uttkrp\_DK = 9.096e+04  
[mode 0] min new rate = 9.097e+04 negatives = 0  
Is G\_DK\_M finite before updating? True  
Are there NaNs in G\_DK\_M after updating cache? False  
Is G\_DK\_M finite after updating? True

Are the shape parameters positive? True  
Are the rate parameters positive? True  
Are the shape parameters finite? True  
Are the rate parameters finite? True  
[mode 1] min uttkrp\_DK = 1.305e+03  
[mode 1] min new rate = 1.305e+03 negatives = 0  
Is G\_DK\_M finite before updating? True  
Are there NaNs in G\_DK\_M after updating cache? False  
Is G\_DK\_M finite after updating? True

Are the shape parameters positive? True  
Are the rate parameters positive? True  
Are the shape parameters finite? True  
Are the rate parameters finite? True  
[mode 2] min uttkrp\_DK = 1.761e+04  
[mode 2] min new rate = 1.761e+04 negatives = 0  
Is G\_DK\_M finite before updating? True

```

Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? True

/home/luiyusen/projects/bptf_new/bptf/src/bptf/bptf.py:218: RuntimeWarning:
invalid value encountered in log
    self.G_DK_M[m] = np.exp(sp.psi(shp_DK) - np.log(rte_DK))

4%|
| 2/50 [00:18<07:21, 9.20s/it]

Are the shape parameters positive? True
Are the rate parameters positive? False
Are the shape parameters finite? True
Are the rate parameters finite? True
[mode 3] min uttkrp_DK = -4.417e-09
[mode 3] min new rate = -4.388e-09 negatives = 1
Is G_DK_M finite before updating? True
Are there NaNs in G_DK_M after updating cache? False
Is G_DK_M finite after updating? False

```

```

-----
AssertionError                                Traceback (most recent call last)
Cell In[1], line 33
    30 # Instantiate and fit the NumPy-based BPTF model.
    31 # Note: This version uses its own preprocess() function and can work
↳with sparse.COO.
    32 model_np = BPTF(data_shape=data_np.shape, n_components=n_components)
--> 33 model_np.fit(data, mask=mask_np, max_iter=50, verbose=True,
↳missing_val=1)
    35 # Reconstruct using arithmetic expectation
    36 reconstruction_np = model_np.reconstruct(mask=None, fill_value=1,
↳drop_diag=False, style='arithmetic')

File ~/projects/bptf_new/bptf/src/bptf/bptf.py:290, in BPTF.fit(self, data,
↳mask, missing_val, **kwargs)
    286     assert is_binary(mask)
    288     self._init()
--> 290     self._update(data, mask=mask, missing_val=missing_val, **kwargs)
    291     return self

File ~/projects/bptf_new/bptf/src/bptf/bptf.py:252, in BPTF._update(self, data,
↳mask, missing_val, modes, **kwargs)
    250     self._update_variational_params(m, data, mask, missing_val)
    251     self._update_beta(m)
--> 252     self._check_mode(m)
    253     bound = self._elbo(data, mask=mask)
    254     delta = (bound - curr_elbo) / abs(curr_elbo)

```

```
File ~/projects/bptf_new/bptf/src/bptf/bptf.py:159, in BPTF._check_mode(self, m)
    157 def _check_mode(self, m):
    158     assert np.isfinite(self.E_DK_M[m]).all()
--> 159     assert np.isfinite(self.G_DK_M[m]).all()
    160     assert np.isfinite(self.shp_DK_M[m]).all()
    161     assert np.isfinite(self.rte_DK_M[m]).all()
```

AssertionError:

```
[ ]: # -----
# 1. PyTorch-based BPTF (your version)
# -----
# Import your PyTorch-based BPTF model (adjust filename as needed)
from own_implementation import BPTF as BPTF_torch
tl.set_backend('pytorch')

device = 'cpu'
# Create a tensor from a Poisson distribution (counts) and a matching mask;
# ↳ ensure types match
data_torch = torch.tensor(data.todense(), dtype=torch.float64, device=device)
mask_torch = torch.ones(expected_shape, dtype=torch.float64, device=device)

# Instantiate and fit the PyTorch-based BPTF model
model_torch = BPTF_torch(data_shape=expected_shape, n_components=n_components,
# ↳ device=device)
model_torch.fit(data_torch, mask=mask_torch, max_iter=50, tol=1e-4,
# ↳ verbose=True)
reconstruction_torch = model_torch.reconstruct(mask=mask_torch,
# ↳ style='arithmetic')
frobenius_diff_torch = torch.norm(data_torch - reconstruction_torch, p='fro').
# ↳ item()
print("PyTorch BPTF reconstruction Frobenius norm difference:",
# ↳ frobenius_diff_torch)
```

```
[ ]: # -----
# 2. TensorLy CP Decomposition
# -----
# Use TensorLy's parafac for CP decomposition (same rank as n_components)
cp_decomp = parafac(data_torch, rank=n_components, n_iter_max=100,
# ↳ init='random')
reconstruction_cp = tl.cp_to_tensor(cp_decomp)
frobenius_diff_cp = torch.norm(data_torch - reconstruction_cp, p='fro').item()
print("TensorLy CP decomposition Frobenius norm difference:", frobenius_diff_cp)
```